

Suivi 1 - Projet Génie Logiciel

Équipe 51 "BRETCH"

19 décembre 2025

Présentation de l'équipe —————

Les compétences personnelles

Caractéristiques personnelles	Ahmed	Akram	Ikhlas	Thibaut	Widad
Leadership	4	5	5	4	3
Planification	3	5	4	4	4
Esprit d'équipe	5	3	4	4	5
Organisation	4	4	4	3	4
Persévérance	5	3	4	5	5
Ponctualité	5	3	3	3	4
Créativité	4	3	4	2	3
Débrouillardise	3	4	4	3	4
Orateur	3	4	3	1	2
Communication écrite	3	2	4	1	4

Les compétences techniques

Caractéristiques techniques	Ahmed	Akram	Ikhlas	Thibaut	Widad
POO	4	4	4	5	3
Modélisation / Conception	4	3	4	4	3
Théorie de langage	3	3	2	4	1
Git	4	5	3	3	3
Jacoco	3	3	1	2	1

La matrice SWOT d'équipe

Forces	Faiblesses
— Leadership et esprit d'équipe développés. — Base solide en POO et en conception. — Utilisation efficace de Git pour le travail collaboratif.	— Lacunes en outils de qualité logicielle (Jacoco). — Fragilité en théorie de langage.
Opportunités	Menaces
— Amélioration rapide via l'apprentissage des tests et de la qualité logicielle. — Meilleure compréhension des compilateurs, parsers, et fondements du génie logiciel	— Projet à distance, — Désavantage face à des profils plus théoriques. — Difficulté en rapport avec le timing du projet

Liste des erreurs pour l'étape B (sans-objet) —————

La suite du document liste les messages d'erreur contextuels (étape B) pour le langage Deca sans-objet. Format des messages :

fichier.deca:ligne:colonne: message.

Erreurs de déclaration et types

Identificateur non déclaré

Code : { int y = x + 1; } (si x non déclaré)

Message : Identificateur "x" non déclaré.

Règle : (0.1) `identifieur` $\downarrow$ env_exp $\uparrow$ def $\rightarrow$ `Identifieur` $\uparrow$ name
avec affectation `def := env_exp(name)`

Type inconnu

Code : { MonType x; }

Message : Type "MonType" inconnu.

Règle : (0.2) `type` $\downarrow$ env_types $\uparrow$ type $\rightarrow$ `Identifieur` $\uparrow$ name
avec condition `(_, type) := env_types(name)`

Variable de type void

Code : { void v; }

Message : Impossible de déclarer une variable de type 'void'.

Règle : (3.17) decl_var ... condition type void

Double déclaration dans même bloc

Code : { int x; float x; }

Message : Déclaration multiple de l'identificateur "x".

Règle : Condition d'unicité dans env_exp

Erreurs d'affectation

Affectation de types incompatibles

Code : { int i; i = 3.14; }

Message : Affectation de type incompatibles : 'float' vers 'int'.

Règle : (3.28) rvalue ↓env_types ↓env_exp ↓class ↓type1 → expr ↓env_types ↓env_exp ↓class ↑type2

avec condition assign_compatible(env_types, type1, type2)

Lvalue invalide

Code : { 3 = x; }

Message : Le membre gauche d'une affectation doit être une variable.

Règle : (3.32) expr ... → Assign [lvalue ... expr ...]

combiné avec (3.64)-(3.69) il faut vérifier que c'est une lvalue

Erreurs dans les conditions

Condition non booléenne dans if

Code : { if (5) { } }

Message : Condition doit être booléenne, type 'int' trouvé.

Règle : (3.29) condition ↓env_types ↓env_exp ↓class → expr ↓env_types ↓env_exp ↓class ↑boolean

Condition non booléenne dans while

Code : { while (42) { } }

Message : Condition doit être booléenne, type 'int' trouvé.

Règle : (3.29) appliqué à l'instruction

(3.25) While [condition ...]

Erreurs dans les opérations arithmétiques

Addition de types incompatibles

Code : { 3 + true; }
Message : Opérateur '+' incompatible avec les types 'int' et 'boolean'.
Règle : (3.33) $\text{expr} \dots \rightarrow \text{op_bin } \uparrow \text{op} \text{ [expr } \uparrow \text{type1 expr } \uparrow \text{type2]}$
avec affectation $\text{type} := \text{type_binary_op}(\text{op}, \text{type1}, \text{type2})$

Opérateur modulo sur flottants

Code : { 3.14 % 2; }
Message : Opérateur '%' exige des opérandes entiers.
Règle : (3.33) avec opérateur 'mod' et $\text{type_binary_op}(\text{mod}, \text{int}, \text{int})$

Opérateur unaire '-' sur boolean

Code : { -true; }
Message : Opérateur unaire '-' incompatible avec type 'boolean'.
Règle : (3.37) $\text{expr} \dots \rightarrow \text{op_un } \uparrow \text{op} \text{ [expr } \uparrow \text{type1]}$
avec affectation $\text{type} := \text{type_unary_op}(\text{op}, \text{type1})$

Opérateur '!' sur non-booléen

Code : { !3; }
Message : Opérateur '!' incompatible avec type 'int'.
Règle : (3.37) avec opérateur 'not' et $\text{type_unary_op}(\text{not}, \text{boolean})$

Erreurs dans les opérations booléennes

Opérateur && sur non-booléens

Code : { 3 && 4; }
Message : Opérateur '&&' exige des opérandes booléens.
Règle : (3.33) avec opérateur 'and' et $\text{type_binary_op}(\text{and}, \text{boolean}, \text{boolean})$

Opérateur || sur non-booléens

Code : { 3 || 4; }
Message : Opérateur '||' exige des opérandes booléens.
Règle : (3.33) avec opérateur 'or' et $\text{type_binary_op}(\text{or}, \text{boolean}, \text{boolean})$

Erreurs de conversion de type

Cast impossible

Code : { (int)("hello"); }

Message : Conversion de type impossible de 'string' vers 'int'.

Règle : (3.39) $\text{expr} \dots \rightarrow \text{Cast } [\text{type} \uparrow \text{type2}]$
avec condition $\text{cast_compatible}(\text{env_types}, \text{type2}, \text{type})$

Instanceof avec types non-classes

Code : { 3 instanceof int; }

Message : 'instanceof' incompatible avec les types 'int' et 'int'.

Règle : (3.40) $\text{expr} \dots \rightarrow \text{InstanceOf } [\text{expr} \uparrow \text{type1} \text{ type } \uparrow \text{type2}]$ avec affectation
 $\text{type} := \text{type_instanceof_op}(\text{type1}, \text{type2})$

Erreurs dans les instructions d'affichage

Print sur type non imprimable

Code : { print(true); }

Message : Impossible d'afficher une expression de type 'boolean'.

Règle : (3.31) $\text{exp_print} \downarrow \text{env_types} \downarrow \text{env_exp} \downarrow \text{class} \rightarrow \text{expr} \downarrow \text{env_types} \downarrow \text{env_exp} \downarrow \text{class} \uparrow \text{type}$
avec condition $\uparrow \text{type} = \text{int}$ ou $\uparrow \text{type} = \text{float}$ ou $\uparrow \text{type} = \text{string}$

Printlnx sur type non numérique

Code : { printlnx("hello"); }

Message : Impossible d'afficher en hexadécimal une expression de type 'string'.

Règle : (3.31) appliqué avec option hexadécimale (même contrainte de type)

Erreurs dans les instructions de retour

Return manquant dans méthode non-void

Code : int f() { }

Message : Instruction return manquante dans méthode non-void.

Règle : (3.24) $\text{inst} \downarrow \text{env_types} \downarrow \text{env_exp} \downarrow \text{class} \downarrow \text{return} \rightarrow \text{Return } [\text{rvalue} \downarrow \text{return}]$
avec condition $\downarrow \text{return} \neq \text{void}$

Type de retour incompatible

Code : `int f() { return 3.14; }`

Message : `Return type mismatch: expected 'int', found 'float'.`

Règle : (3.24) combiné avec

(3.28) pour la compatibilité de type

Restrictions spécifiques sans-objet

Utilisation de 'this' en programme principal

Code : `{ this; }`

Message : `'this' ne peut pas être utilisé dans le programme principal.`

Règle : (3.43) `expr ... → This` avec condition `class != 0`

Erreurs d'initialisation

Initialisation incompatible

Code : `{ int x = "hello"; }`

Message : `Impossible d'initialiser une variable de type 'int' avec expression de type 'string'.`

Règle : (3.8) `initialization ... → Initialization [ rvalue ... ]` combiné avec (3.28)

travail pas encore fait

Erreurs contextuelle :

- new sur classes
- vérification de subtype
- Gestion des classes (objet)

Partie A,B,C

- implemnatation de ces erreurs
- Lien avec Le syntax (Lexer , Parser)
- Lexer complet
- Parser complet et fonctionnel
- Arbre de decision colorée
- Generation de code Assembly

En Resume —————

Ce qui est vérifié par l'analyseur syntaxique (étape A)

- Structure syntaxique (parenthèses, point-virgules, guillemets non fermés, etc.)
- Mots-clés interdits en sans-objet (`new`, `this`, `null`, `instanceof`, `class`, `extends`, `protected`)
- Appels de méthode (interdits en sans-objet sauf `print`, `println`, `printx`, `printlnx`, `readInt`, `readFloat`)
- Syntaxe correcte des expressions selon la grammaire concrète
- Vérification que le membre gauche d'une affectation est syntaxiquement une lvalue

Ce qui est vérifié par l'analyseur contextuelle (étape B)

- Déclarations : identificateurs déclarés, types existants, pas de void, pas de doublons
- Typage : compatibilité d'affectation, opérateurs sur bons types, conversions valides
- Conditions : if/while avec booléens, return présent dans méthodes non-void
- Affichage : print sur int/float/string seulement
- Contextes : this seulement dans méthodes, new sur classes
- Sous-typage : vérification de subtype, `assign_compatible`, `cast_compatible`